

Side-channel Leakage in Low Entropy Masking Schemes Attributed to FPGA Architecture

Yanbin Li, Yikang Guo, Fan Zhang

Abstract—Masking is one of the most popular to guarantee the security of the cryptographic implementation against side-channel analysis. Low Entropy Masking Scheme (LEMS) has been proposed to relieve the high overhead by reducing the entropy of the mask. The masks are selected carefully to resist univariate first-order attacks. However, in practice, securely implementing masking can be a challenging task. In this work, we exhibit the vulnerability of LEMS implementation on FPGA. Firstly, we provide the security model to characterize the masking balance at the implementation level. Based on the security model, we exhibit a first-order leakage in the netlist of the implementation. It can be proved that this defect is due to the uncontrollable EDA tool to optimize the AES Sbox implementation on the specific architecture of FPGA during synthesis. The discovered flaw can be exhibited by performing a couple of first-order attacks to recover the secret key successfully on FPGA. The vulnerability is verified by simulation and practical measurements. Finally, the potential countermeasure and security evaluation process for LEMS are provided to avoid leakage during the pre-silicon design phase.

Index Terms—Side-channel Analysis, AES, Low Entropy Masking Scheme, FPGA.

I. INTRODUCTION

SIDE-CHANNEL analysis (SCA) has emerged as a serious threat to the cryptographic algorithms by exploiting the physical leakage. It includes the running time, power consumption, or electromagnetic radiation during the cryptographic algorithm execution. Due to the threat to ASIC, and FPGA implementation of the cryptographic algorithm, many different countermeasures have been considered and developed.

Masking is a well-known effective countermeasure to protect the implementations of cryptographic algorithms against SCAs. The idea of masking is to introduce random numbers to make the observable physical leakage independent of sensitive information. A provable security model was proposed by Ishai et al. [1], which assumed that any d intermediate variables could be probed in the masking scheme. Later, there were many higher-order masking schemes proposed to achieve d th-order security [2], [3], [4], [5], [6]. Along with the higher security, the overhead of a higher-order masking scheme is non-negligible and implementing it in the constrained device is a challenging task.

Y. Li is with the School of Software, Shandong University, Jinan, China, 250100, with the State Key Laboratory of Cryptology, P.O.Box 5159, Beijing, China, 100878.

Y. Guo is with the College of Artificial Intelligence, Nanjing Agricultural University, Nanjing, China, 210095.

F. Zhang is with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, 310007, with the Alibaba-Zhejiang University Joint Institute of Frontier Technologies, Hangzhou, China, 311121.

To seek the trade-off between security and overhead, reducing the entropy of the mask has been utilized to relieve the high overhead. This type of masking scheme is called the low-entropy masking scheme (LEMS) [7], [8], [9]. Compared to generating masks randomly from $GF(2^n)$ in full-entropy masking schemes (FEMS), the masks are selected from the subset of $GF(2^n)$ in LEMS. The Rotating Sbox Masking (RSM) was proposed based on the look-up table (LUT-based) [7], aiming at resistance of first-order Differential Power Analysis Attack (DPA)/ Correlation Power Analysis Attack (CPA) with low overhead. RSM was also adopted as the countermeasure of AES (Advanced Encryption Standard) in DPA Contest v4 [9].

The security of RSM and its variants [9], [10], [11], [12], [13] has been investigated for a long time. In the implementation of RSM, the input mask of Sbox m_i and the output mask m_{i+1} share the same register, which makes the power consumption related to $m_i \oplus m_{i+1}$. This unbalanced feature was exploited by Moradi et al. to perform the univariate correlation power attack [14]. Ye et al. proposed the leakage distribution decomposition attack and the leaking set collision attack to retrieve the secret key in LEMS [10]. Grosso et al. illustrated that the higher-degree terms in the leakage function influenced the security order of LEMS [15]. Besides, performing the first-order attack combining multiple bits recovered the secret key successfully [13]. The importance of a proper mask set in LEMS was analyzed and summarized in [9].

While research on LEMS mainly focused on the balance of mask set and leakage distribution, few efforts analyzed the security of the masking implementation on FPGA. Moradi et al. analyzed the security of DPA-resistant logic styles on FPGA and provided solutions for imbalanced routing and early propagation [16]. The suitability against first-order attacks by randomized look-up tables was evaluated under different Xilinx FPGA memory primitives [17]. So far, there is no work on the security of LEMS on the specific FPGA architecture.

The AES Sbox can be considered an 8-bit input and 8-bit output Boolean function. In RSM, the masked Sbox is implemented as a look-up table with 8 inputs and 8 outputs, which is transformed from the original Sbox according to the mask value. However, the maximum support is 6-input LUT in modern FPGAs, including Xilinx Virtex-5,6,7, and Spartan-6. Regarding look-up table-based FPGA implementations, the constructions of the masked Sbox should be optimized to map the available slices on FPGA. The security of LEMS during the optimization by uncontrollable FPGA tools has not been discussed.

This work belongs to solving the security gap between

schemes and implementations. Securely implementing masking can be a challenging task. Several works have shown that the glitch could make the masking scheme fail to achieve its security order in the masked circuits [18], [19], [20]. The masked gates were vulnerable to first-order attacks because of the glitches [18]. The toggle-count model was used as the leakage model to recover the secret key targeting the masked Sbox [19], [20]. Moreover, another type of leakage occurs during synthesis optimization by standard Electronic Design Automation (EDA) tools [21]. As in [21], the implementation of SecMult algorithm had a pitfall after synthesis [2]. The masking circuit was implemented by 4-input LUTs that leaked the sensitive information in the output of 4-input LUT. The security flaw found in this work is also related to the EDA optimization on the FPGA architecture.

In this work, we present the vulnerability of look-up table-based LEMS on FPGA for the first time, which is influenced by the inherent specific architecture. The security is investigated from the perspective of the balance of the mask set. Even if the mask set is balanced in a masking scheme, it may not achieve the security order at the implementation level. The main contributions of this work include:

- **The Bit-Balance (BB) model was proposed in [22] to characterize the balance of the mask set. We extend it to the hardware implementation, named *Bit-Balance Implementation (BBI)* model, to evaluate the security at the implementation level.**
- The vulnerability of the FPGA implementation of RSM is exhibited under *BBI* model. We analyze the discovered flaw that occurred by the uncontrollable EDA tool to optimize the AES Sbox implementation on the specific architecture of FPGA during synthesis.
- Experiments with simulation and practical measurements from the FPGA implementation of RSM are conducted to verify the security flaw. Moreover, the potential countermeasure and security evaluation process for LEMS are provided to avoid leakage during the pre-silicon design phase.

The remainder of this paper is organized as follows. Section 2 demonstrates the definitions in LEMS and describes the RSM scheme. Besides, we show the power consumption model of FPGA, which is used in the following security analysis. Section 3 provides *BBI* model and shows the potential vulnerability in LEMS on FPGA. The simulation and practical attacks on FPGA are provided in Section 4. The discussion of possible countermeasures and security evaluation process is provided in Section 5. Section 6 concludes the paper.

II. PRELIMINARIES

A. Low Entropy Masking Schemes (LEMS)

To reduce the high cost of masking to resist side-channel attacks, LEMSs are designed to maintain the security of the cryptographic implementation. Like other masking schemes, it draws masks from a subset of $GF(2^n)$ to randomize the intermediate value, where n denotes the bit-length of the intermediate variable in a cryptographic algorithm.

Compared with the Full Entropy Masking Schemes (FEMS), protecting the sensitive information with the masks from the entire set, LEMS saves lots of repeated computation for the non-linear operation. For a well-designed LEMS implementation, its performance and computational complexity are acceptable to the practical application. The definitions of the full entropy mask set and low entropy mask set are also given in [13].

Definition 1 (Full Entropy Mask Set) Let $\mathbb{M}$ be an n -bit mask set and M be the mask variable. $H(M)$ denotes the entropy of M , where $M \in \mathbb{M}$. $\mathbb{M}$ is a full entropy mask set if $H(M) = n$.

Definition 2 (Low Entropy Mask Set) Let $\mathbb{M}$ be an n -bit mask set and M be the mask variable. $H(M)$ denotes the entropy of M , where $M \in \mathbb{M}$. $\mathbb{M}$ is a low entropy mask set if $H(M) \in (0, n)$.

Generally, the complexity of masking operations in LEMS is lower because it needs less randomization than FEMS. Resisting side-channel attacks such as CPA and DPA requires carefully designing the scheme. Since the masks in the mask set are selected from a subset, one critical condition to achieve first-order security is that each bit of the masked variables should be independent of the unmasked bits.

B. Rotating Sboxes Masking Scheme (RSM)

Rotating Sboxes Masking (RSM) and its variants [7], [8], [9] are typical of the LEMS implementations. It can resist first-order SCA attacks, second-order SCA attacks, and other attacks at the cost of consuming a small amount of area and performance by limiting the value range of the random mask to 16 fixed elements. In order to guarantee security, it is necessary to achieve balance when choosing a random mask in the fixed set. Each bit of the mask state has the same probability of being equal to 0 and 1.

For the implementation of AES-128, it needs $l = 10$ rounds of encryption. It selects 16 values in the mask set, noted as

$$\mathbb{M} = \{m_0, m_1, m_2, m_3, m_4, m_5, m_6, m_7, m_8, m_9, m_{10}, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}\} \quad (1)$$

RSM rotates Sboxes with these masks for the entire computation of the AES algorithm. The 16 masked Sboxes are pre-computed by

$$MSbox_i(X) = S(X \oplus M_i) \oplus M_{i+1} \quad (2)$$

The masks are determined by the random offset drawing from $[0, 15]$ at the beginning of the computation. The datapath of RSM implementation is shown in Fig. 1. It uses *MaskCompensation* to unmask the data and re-mask it to ensure correctness simultaneously. The implementation performs the masking transformations on other operations, such as ShiftRows and MixColumns. For these linear properties, it only requires a simple XOR operation. Finally, it can obtain the ciphertext utilizing the masking scheme.

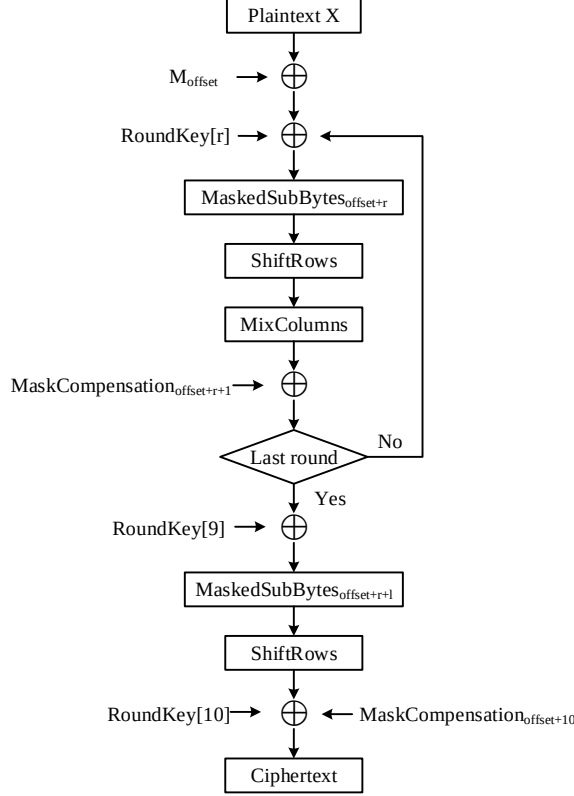


Fig. 1: The RSM datapath [7].

C. Power Consumption Model of FPGA

FPGA logic is built on the Complementary Metal-Oxide Semiconductor (CMOS) transistor pairs at the circuit level. Thus the power consumption of an FPGA satisfies the power characterize of a CMOS circuit.

The power consumption of CMOS includes static power and dynamic power. The switching power plays an important role in dynamic power of the circuit. The main current power simulation tools integrate the power calculation of the design, such as PrimePower and PrimeTime PX. The switching power P_c can be calculated [23] for every rising transition and divided by the total time

$$P_c = \frac{V_{dd}^2}{2} \cdot \sum_i (C_{load_i} \cdot TR_i) \quad (3)$$

where V_{dd} represents the supply voltage, C_{load_i} and TR_i represent the capacitive load and toggle rate of net i , respectively.

In the side-channel area, it had been simplified as the total number of signal toggles in a circuit, named the toggle-count model. It has been exploited to attack the unmasked and masked AES Sbox successfully [20], [24], [25]. Based on equation (3), the toggle-count model represents the crucial factor of data-dependent power, which is used as the power consumption model of FPGA in this work.

III. SECURITY FLAW OF LEMS ON FPGA

In this section, we first summarise the related attacks on the mask sets of RSM. Then recall the *Bit-Balance Security* model [22] and extend it to the hardware implementation. Then, we show the FPGA architecture and provide the AES Sbox implementation of FPGA. Finally, the leakage in the FPGA implementation of RSM based on the security model is presented.

A. Security Model of LEMS

The balance of the mask set is the essential condition to resist the first-order CPA. The mask set is continuously updated and improved as research progresses.

$$\begin{aligned}
 MaskSet_1 &= \{0x00, 0x0f, 0x36, 0x39, 0x53, 0x5c, 0x65, \\
 &\quad 0x6a, 0x95, 0x9a, 0xa3, 0xac, 0xc6, 0xc9, 0xf0, 0xff\} \\
 MaskSet_2 &= \{0x03, 0x0c, 0x35, 0x3a, 0x50, 0x5f, 0x66, \\
 &\quad 0x69, 0x96, 0x99, 0xa0, 0xaf, 0xc5, 0xca, 0xf3, 0xfc\} \\
 MaskSet_3 &= \{0x13, 0x94, 0x25, 0xcb, 0x8e, 0x5f, 0xd9, \\
 &\quad 0x37, 0x77, 0xc6, 0xa8, 0x38, 0x05, 0xea, 0x70, 0xe8\} \\
 MaskSet_4 &= \{0x00, 0x0f, 0x36, 0x39, 0x53, 0x95, 0x5c, \\
 &\quad 0xc9, 0xff, 0xc6, 0xac, 0x9a, 0x6a, 0xa3, 0x65, 0xf0\}
 \end{aligned} \quad (4)$$

Currently, there are four mask sets for RSM schemes. The first mask set $MaskSet_1$ was proposed in DPA contest v4.1 [9], however, the subsequent works showed it can not resist the first-order CPA attack for the vulnerabilities in the mask set [14], [26]. The $MaskSet_2$ was used in DPA contest v4.2, which was attacked by exploiting a bivariate vulnerability [11]. The $MaskSet_3$ proposed by Veshchikov et al. [26] and $MaskSet_4$ proposed by Moradi et al. [14] had not been attacked, until Ming et al. showed the attacks exploiting the multivariable leakage [13]. It exhibited that there is no secure mask set for RSM to achieve the expected security level under multivariable leakage assumption.

In this work, we don't focus on the bivariate, multivariable leakage assumption or second-order attacks. This work primarily reveals the leakage caused by the FPGA architecture, which can be demonstrated even under the simplest univariate leakage model, i.e. single-bit model or Hamming weight model. This section takes the $MaskSet_4$ as example.

The *Bit-Balance Security* model was defined to evaluate the first-order bit-balance security of the mask set in LEMS [22]. It also considered the univariate leakage model. The $MaskSet_4$ is validated and considered to resist the first-order DPA/CPA.

Definition 3 (Bit-Balance Security [22]) Assume M is the random variable of mask in $\mathbb{M}$, X is the n -bit unmasked variable and X_M represents the masked variables after masking operation. The unmasked variable X is uniformly distributed over $GF(2^n)$. $X[k]$ and $X_M[k]$ denote the k -th bit of its variable, where $k \in [0, n-1]$. $\forall k \in [0, n-1]$, if $E(X_M[k]|X[k] = 0) = E(X_M[k]|X[k] = 1)$, the masking scheme with mask set $\mathbb{M}$ is Bit-Balance (BB) secure.

The *Bit-Balance Security* model reflects the balance of masked variables at the algorithm level. In the actual attack,

there is always a gap between the algorithm and practical implementation. We extend the *Bit-Balance Security* model to the hardware implementation scene.

The implementation of LEMS is designed and considered to resist first-order DPA/CPA. Assume an implementation of the cryptographic algorithm, Q is the input or output and S represents the secret key. The sensitive information is $X = f(Q, S)$, where f denotes the crucial function in a cryptographic algorithm. In the masked implementation, the mask M is generated from a set $\mathbb{M} = \{m_0, m_1, \dots, m_{|\mathbb{M}|}\}$ so that the observable leakage L only relates to the masked information $X_M = \{f_M(Q, S)\}$, where the masked function f_M should be designed and implemented carefully. The power consumption L can be represented as:

$$L = H(X_M) + \varepsilon = H(f_M(Q, S)) + \varepsilon \quad (5)$$

where $H(\cdot)$ represents the leakage model, ε denotes a gaussian noise.

The observable leakage L from the implementation should be independent of X . The statistic dependency between L and X can be described as:

$$L \perp X \implies \text{Var}[E(L|X)] = 0 \quad (6)$$

where $\perp$ represents the statistical independence, E represents the expectation and Var represents the variance. The expectation is computed by the mean of L for each class x taken by X . The variance measures the inter-class variance of L in each class $X = x$. The CPA calculates the Pearson correlation coefficient between the sensitive information and observable leakage. When $\rho(L, X) = \rho(E(L|X), X) \neq 0$, the $E(L|X = x)$ does not vary with the value x . That means the inter-class variance of the expectation is zero, which can resist the CPA attack. For a countermeasure, the balance of masking implementation should be satisfied to avoid inter-class variance, i.e., $\text{Var}[E(L|X)] = 0$ [27]. In the following, the *Bit-Balance Security* model is extended to the hardware implementation of LEMS.

Definition 4 (Bit-Balance Implementation Security)

Assuming f is the function that outputs the masked variable X_M , circuit C_f implements the function. The unmasked variable X is uniformly distributed over $GF(2^n)$. $X[k]$ and $X_M[k]$ denote the i th-bit of its variable, where $i \in [0, n-1]$. $\forall k \in [0, n-1]$. Let H be the leakage model of the circuit C_f . If $E(H(X_M[k])|X[k] = 0) = E(H(X_M[k])|X[k] = 1)$, the masking scheme with mask set $\mathbb{M}$ is *Bit-Balance Implementation (BBI) secure*.

BBI model implies the implementation security in the perspective of bit balance to resist first-order CPA/DPA.

B. FPGA architecture

FPGA is the re-configurable hardware consisting of configurable logic blocks named CLB. The CLB is the basic building block containing logic cells, shown in Fig. 2 (a). The CLBs are connected through programmable interconnects of wire segments. They are organized into rows and columns and assigned coordinate values.

The CLB of Modern Xilinx FPGAs contains two slices. In Fig. 2 (b), it can be found that the slice includes four look-up tables (LUTs), carry logic and registers. The LUT is the basic building block that can be used to implement logic functions. It implements different functions to output values by the truth table with different combinations of the inputs.

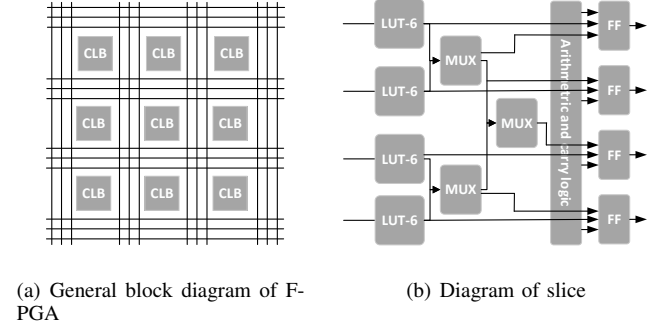


Fig. 2: CLB Arrangement and slice description.

For a general N -input LUT, the number of addresses accessed by the truth table is 2^N , where N represents the number of inputs. The traditional Spartan-3 and Virtex-4 FPGAs maximum support 4-input LUTs, while modern FPGAs support 6-input LUTs, for example Spartan-6, Virtex-5,6 and 7.

AES is a well-known symmetric cryptographic algorithm. By default, it refers to AES-128 in this paper. It requires ten rounds, including Sbox, ShiftRows, MixColumns and AddRoundKey. The Sbox is the most consuming operation. The input size and output of Sbox are 8 bits. Its look-up table implementation needs $2^8 = 256$ input addresses to map the correct output value. However, the mainly modern FPGAs maximum support 6-input LUTs. Instead, its Sbox implementation of FPGA uses the 6-input LUTs combined with multiplexors [28]. The implementation structure of Virtex-5 can be seen in Fig. 3.

Let $x = x_7x_6x_5x_4x_3x_2x_1x_0$ represents the input of Sbox operation and $y = y_7y_6y_5y_4y_3y_2y_1y_0$ represents the output. We take the implementation of output y_0 as an example. Fig. 3 shows the implementation of calculation the output bit y_0 . Theoretically, one byte Sbox can be implemented by eight pieces of the circuit. The structure for 8 bits is the same, while the truth tables in the LUTs are different. It needs $2^8 = 256$ addresses in a look-up table, replaced by four 6-input LUTs. Moreover, the bit x_6 and x_7 are used as selection signals in multiplexors to indicate the correct output.

Instead of a simple implementation of one 8-input LUT, it is implemented by a more complex circuit including four LUTs and three multiplexors. The power consumption of this circuit is related to the activities of the components. It only restricts the output signal y_0 to be balanced under BB model. Obviously, the characteristics of the whole circuit need to be considered in the actual implementation of FPGA.

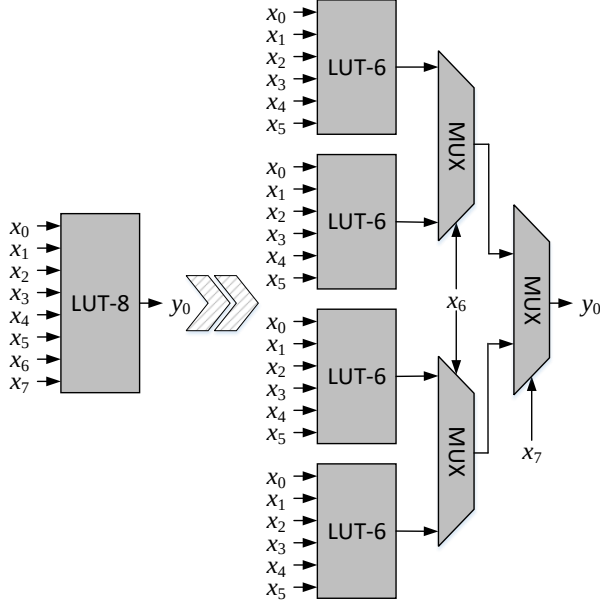


Fig. 3: The implementation of AES Sbox on FPGA. $y_7y_6y_5y_4y_3y_2y_1y_0 = Sbox(x_7x_6x_5x_4x_3x_2x_1x_0)$. This figure represents the output y_0 . The structure for other bits are the same with y_0 .

C. Leakage analysis

RSM carefully chooses the mask values to achieve *Bit-Balance Security*. However, this security stays at the algorithm level. The power character is related to the underlying implementation.

The RSM scheme is implemented on Xilinx Spartan-6 by ISE 14.4. The implementation of the masked Sbox is shown in Fig. 4. The 16 masked Sboxes are pre-computed according to equation (2). The masked Sbox is implemented in the combinational logic within one clock cycle. Each of them is an 8-to-8-bit function, which can be considered eight 8-to-1-bit functions. During the synthesis of EDA tool, the inputs of LUTs for the masked Sbox are replaced by the bits of the masked variable and the truth tables are initialed corresponding to the masked Sbox automatically. Each 8-to-1-bit function is synthesized into four 6-input LUTs occupying one slice. The 8-to-1-bit look-up table of masked Sbox requires 32 LUTs, all fitting into 8 slices.

To analyze the leakage of the implementation, it needs to access the detailed description of the circuit. The implementation details can be found in the netlist at the place & route level. There are 16 masked Sboxes during one round total. The 16 masked Sboxes are synthesized into two structure modes.

The first mode of implementation structure is shown in Fig. 5. The $\backslash aes_unit/add_roundkey0$ is the input of masked Sbox, representing the masked state after the plaintext XOR key. In the implementation of 1st-byte masked Sbox, the six bits $\backslash aes_unit/add_roundkey0[5:0]$ are used as the inputs of 6-input LUTs, while $\backslash aes_unit/add_roundkey0[6]$ and $\backslash aes_unit/add_roundkey0[7]$ are the select signals for the

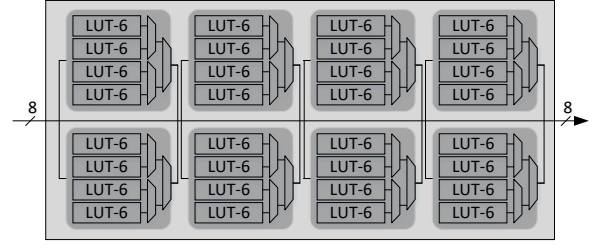


Fig. 4: The implementation architecture of the masked Sbox of RSM.

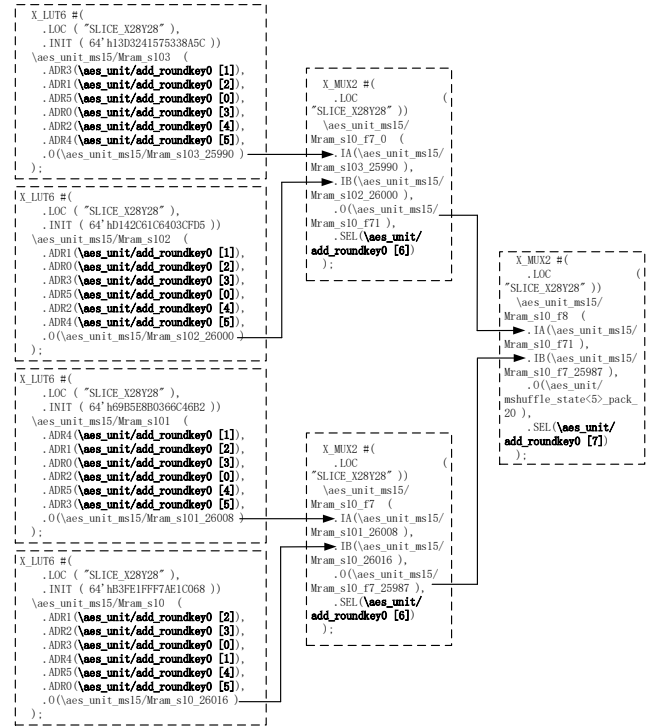


Fig. 5: The first mode of implementation structure of the masked Sboxes.

multiplexors. Through analyzing the netlist, it can be found that there are 14 bytes masked Sboxes implemented in the first mode, i.e. the low six bits are used as the inputs of LUTs and the high two bits are the select signals for the multiplexors.

Another mode is the irregular implementation of 2nd-byte and 13th-byte masked Sboxes. The implementation of the 2nd-byte masked Sbox is shown in Fig. 6. The high six bits $\backslash aes_unit/add_roundkey0[15:10]$ of the masked state are used as the inputs of 6-input LUTs, while the two remaining bits $\backslash aes_unit/add_roundkey0[8]$ and $\backslash aes_unit/add_roundkey0[9]$ are the select signals for the multiplexors. This difference in implementations may occur by uncontrollable EDA tool, which aims at design optimization. It can be found that the 13th-byte masked Sbox takes the $\backslash aes_unit/add_roundkey0[98]$ and $\backslash aes_unit/add_roundkey0[99]$ as the select signals while

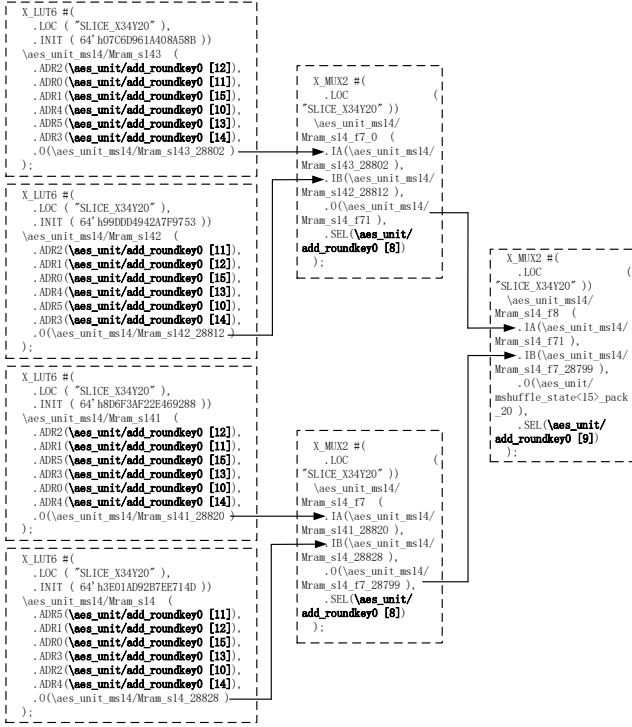


Fig. 6: The second mode of implementation structure of the masked Sboxes.

the remaining six bits as the input of LUTs.

There are 16 masked Sboxes in the RSM scheme. The masked input of masked Sbox $x[0 : 7]$ is randomized by a uniformly distributed random number. The first mode uses low bits $x[0 : 5]$ as the input of LUT-6 and high bits $x[6 : 7]$ as the select signals for the multiplexors. On the contrary, the second mode uses high bits $x[2 : 7]$ as the input of LUT-6 and high bits $x[0 : 1]$ as the select signals for the multiplexors, shown in Fig. 7. The bold lines indicate the select signals. Similarly, 13th-byte masked Sbox takes $x[2 : 3]$ as the select signals and the other bits as the input of LUT-6. The inconsistent structure causes differences in internal states.

The initial truth table of LUTs can be designed by the inputs to guarantee the correct output of the masked Sbox. Thus those different implementations cannot destroy the correctness. However, the composition of the circuit under different masks has changed, which will also influence the power consumption of the masked circuits. Proposition 1 is proposed to discuss how it affects the security of LEMS.

Proposition 1 Assume circuit C_f implements the output of masked Sbox bit $X_M[k]$ by taking X_{in} as the input signals, $k \in [0, n - 1]$. The unmasked variable X is uniformly distributed over $GF(2^n)$. In the circuit of i th-masked Sbox, the bits X_{LUT}^i are used as the input signals of 6-input LUT and $X_{MUX}^i = \{X_{MUX1}^i, X_{MUX2}^i\}$ are used as the select signals for the two multiplexors, i.e. $X_{in}^i = X_{LUT}^i \cup X_{MUX}^i$. If there exists different implementation structures of 16 masked Sbox, i.e. $\exists i, j, X_{LUT}^i \neq X_{LUT}^j, X_{MUX}^i \neq X_{MUX}^j$, the BB secure masking scheme cannot achieve BBI security.

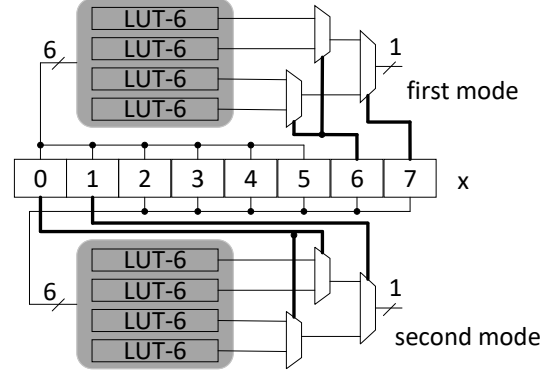


Fig. 7: The inconsistent structure of the masked Sbox.

Proof: Based on the toggle-count model, the power consumption $H(X_M[k])$ is related to the transitions in the circuit:

$$H(X_M[k]) = \sum_{r=0}^3 t_{LUT}^r + \sum_{r=0}^2 t_{MUX}^r$$

where t_{LUT}^r represents the toggle of r th-LUT and t_{MUX}^r represents the toggle of r th-multiplexor.

The output toggle of four 6-input LUT ($LUT_0, LUT_1, LUT_2, LUT_3$) can be represented as

$$\begin{aligned} t_{LUT}^0 &= LUT_0(X_{LUT}) = MSbox(00|X_{LUT}) \\ t_{LUT}^1 &= LUT_1(X_{LUT}) = MSbox(01|X_{LUT}) \\ t_{LUT}^2 &= LUT_2(X_{LUT}) = MSbox(10|X_{LUT}) \\ t_{LUT}^3 &= LUT_3(X_{LUT}) = MSbox(11|X_{LUT}) \end{aligned}$$

where $|$ denotes the bit concatenation and $00|X_{LUT}$ is the input 8 bits of the masked Sbox. The output toggle of the multiplexor can be represented as:

$$\begin{aligned} t_{MUX}^0 &= If_Then_Else(X_{MUX1} = 0, LUT_0(X_{LUT}), \\ &\quad LUT_1(X_{LUT})) \\ t_{MUX}^1 &= If_Then_Else(X_{MUX1} = 0, LUT_2(X_{LUT}), \\ &\quad LUT_3(X_{LUT})) \\ t_{MUX}^2 &= If_Then_Else(X_{MUX2} = 0, t_{MUX}^0, t_{MUX}^1) \end{aligned}$$

$If_Then_Else()$ function returns the former value if the condition is TRUE, or the latter value if the condition is FALSE.

$$If_Then_Else(a, b, c) = \begin{cases} b & \text{if } a = TRUE \\ c & \text{if } a = FALSE \end{cases}$$

Consider the situation corresponding to the inconsistency structures of masked Sbox implementations. $\exists i, j, X_{LUT}^i \neq$

$X_{LUT}^j, X_{MUX}^i \neq X_{MUX}^j$, it has

$$E(t_{LUT}^0|X[k]=0) \neq E(t_{LUT}^0|X[k]=1)$$

$$E(t_{LUT}^1|X[k]=0) \neq E(t_{LUT}^1|X[k]=1)$$

$$E(t_{LUT}^2|X[k]=0) \neq E(t_{LUT}^2|X[k]=1)$$

$$E(t_{LUT}^3|X[k]=0) \neq E(t_{LUT}^3|X[k]=1)$$

Thus,

$$E(H(X_M[k])|X[k]=0) \neq E(H(X_M[k])|X[k]=1)$$

■

In the above FPGA implementation of LEMS, the structures of the 2nd-byte and 13th-byte masked Sboxes differ from others. Based on proposition 1, it cannot achieve *BBI* security. The difference of conditional expectation (DCE) can be used to validate the bias:

$$DCE = E(H(X_M[k])|X[k]=1) - E(H(X_M[k])|X[k]=0) \quad (7)$$

The difference result should be equal to 0 for a masking implementation against univariate first-order attacks. For each bit $k \in [0, 7]$, the DCE can be obtained based on the given implementation. The DCE for each bit is shown in Fig. 8. It exhibits the leakage of FPGA implementation of LEMS under *BBI* model. The results are consistent with Proposition 1.

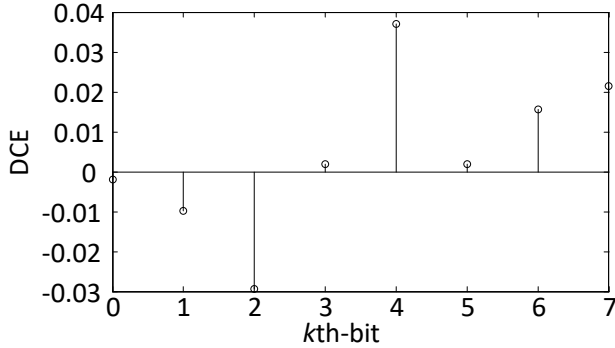


Fig. 8: The DCE for each bit.

IV. ATTACKS EXPLOITING THE FPGA LEAKAGE

A. Simulation analysis

To verify the vulnerability, the side-channel attack experiments are performed on the FPGA implementation of RSM by exploiting the leakage. The experiments use Xilinx ISE design suite 14.4 to design the hardware implementation of RSM. The experiments are conducted on a 1.8GHz Intel CPU computer.

The regular steps of ISE design include design entry, synthesis, implementation, verification and download. The Hardware Description Language (HDL) of RSM scheme and TESTBENCH file are used to perform the simulation. During the design process, the readable netlist file NETLIST.V is generated, which contains detailed information about the circuit. Firstly, we analyze the implementation structure of the masked

Sbox in the netlist. It can be found there are 14 bytes masked Sboxes are implemented in the way of low six bits are used as the input of LUTs and the high two bits are the select signals for the multiplexors (shown in Fig. 5), while the remaining 2 bytes masked Sboxes are not (shown in Fig. 6). According to Proposition 1, the inconsistent structure would lead to leakage of low-entropy masking on FPGA.

Secondly, the leakage by performing the power analysis attacks is verified on the simulation results. We run the simulation of RSM design by ISE Simulator, which is a supported logic simulator in Xilinx ISE platform. The waveform data of signals is dumped to Value Change Dump (VCD) file. In the TESTBENCH, there are 5,000 ($> 256 * 16$) random plaintexts to be encrypted with a fixed secret key. The number of traces covers the space of random plaintext and masks. After performing post-translate simulation, 5,000 simulation results are generated in the VCD file. The toggle-count model is utilized to calculate the simulation power consumption of the circuit.

The sensitive value is calculated according to the original Sbox. The first-order CPA attack is performed on the FPGA implementation. In this group of experiments, we perform the attacks with Hamming weight (HW) model and single-bit model separately. Remarkably, the implementation of RSM should be able to resist the first-order CPA or DPA theoretically [8]. However, the correct key is retrieved by the first-order CPA. That means the FPGA implementation of RSM cannot achieve first-order security under *BBI* model. It can be found in Fig. 9 (a), the black line indicates the correct key, while the grey lines indicate the wrong keys. The correct key is distinguished under Hamming weight model after 2,000 traces.

In addition, we perform the CPA with the single-bit model. The difference of correlation coefficients among guessing keys is shown in Fig. 9 (b)~(i). It can also retrieve the correct key under some single bits. The significant leakage can be observed in the results of bit 1, 2, 4. The difference among different bits may occur by the various delays and the noise's influence. Adopting an orthogonal array mask set is necessary to balance the masked variables in LEMS. The orthogonal array mask set is adopted in this FPGA implementation. However, the bias structure implementation makes it fail to satisfy *BBI* security.

B. Practical Analysis

The target implementation of RSM is described in Verilog and compiled with Xilinx ISE for a Spartan-6 FPGA. We download it onto the SAKURA-G board [29]. The plaintexts and keys are sent from the computer to the target board through a USB port.

The oscilloscope DSOX3024T is employed to capture and transmit the power traces. Fig. 10 depicts the power measurement setup to the FPGA core. The voltage of the main FPGA can be monitored through the SMA connector. The power traces are pre-treatment by PA303 Amplifier. We capture the power traces at a sampling rate of 1.25 GSa/s and a bandwidth limit of 20 MHz while the board runs at a clock frequency of 3 MHz.

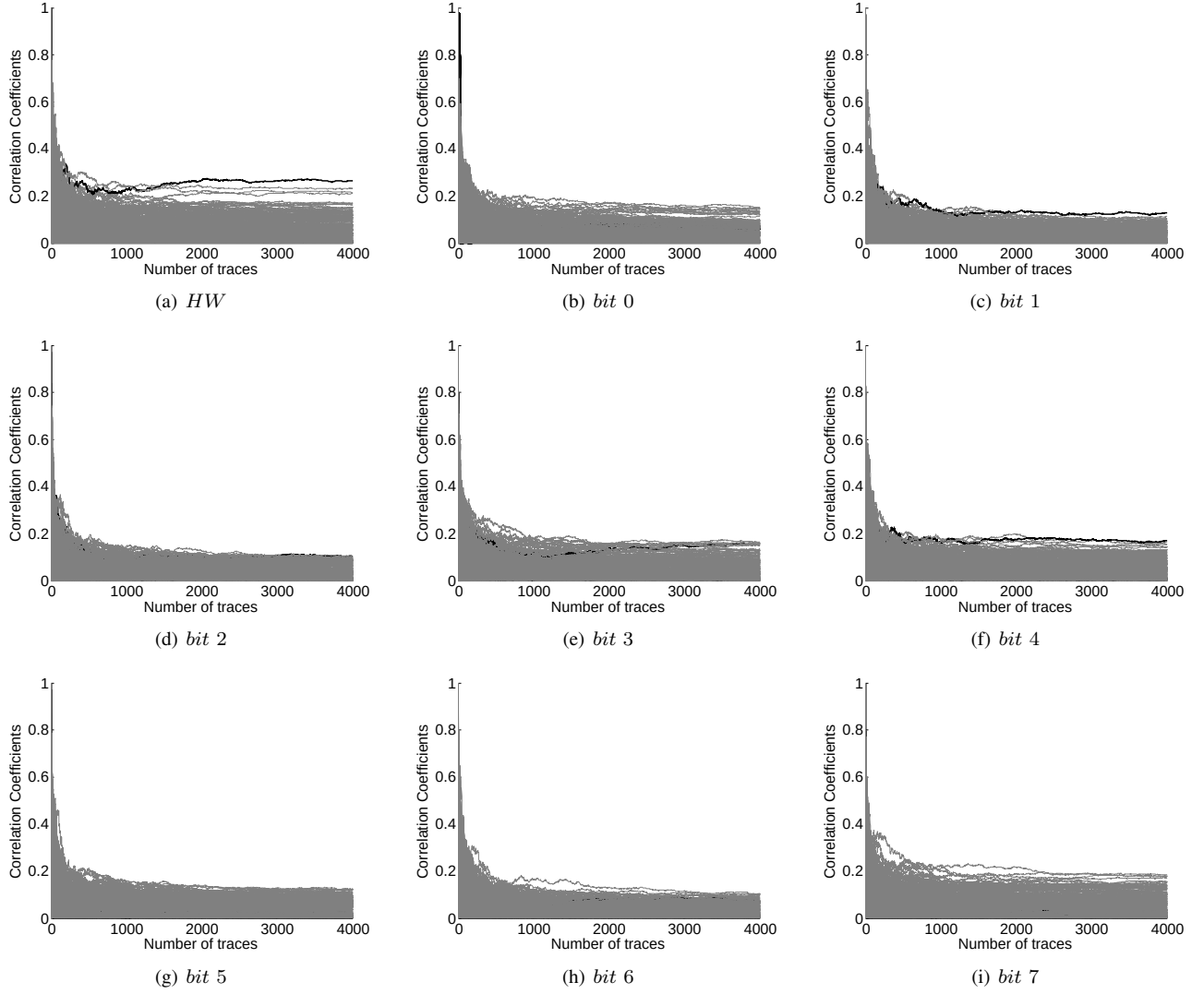


Fig. 9: Attacking results of CPA

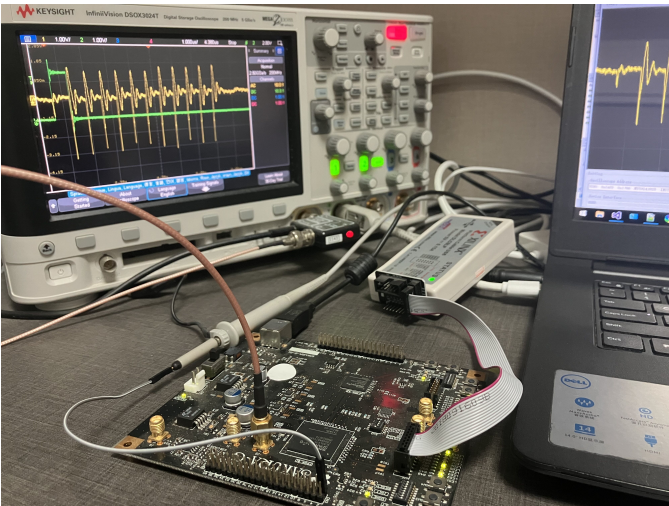


Fig. 10: The complete measurement setup.

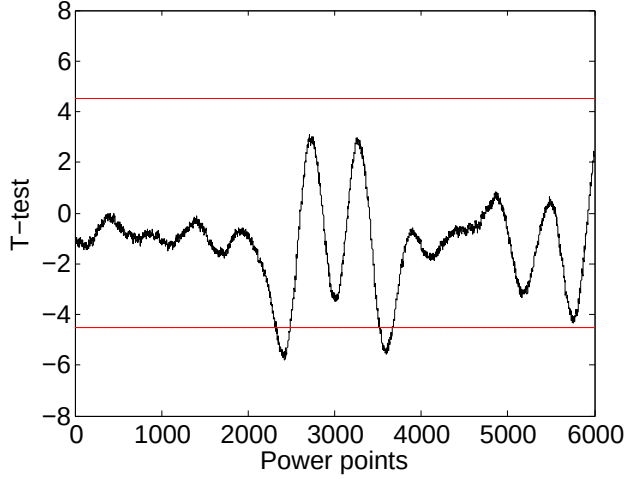
The leakage assessment methodology [30], [31] is exploited to examine whether the RSM's FPGA implementation leaks sensitive information. This method can evaluate the security without attacks relying on Welch's t -test. The most general form of the specific t -test evaluates the power traces of random plaintexts with a fixed key. The captured measurement set is split into two groups according to the intermediate value and leakage model. It examines a null hypothesis that those two groups of data are significantly different from each other.

In this practical experiment, the measured traces are partitioned into two groups L_0 , L_1 according to the sensitive variable, e.g. the output of Sbox. The number of traces in the two sets is denoted as $|L_0|$ and $|L_1|$.

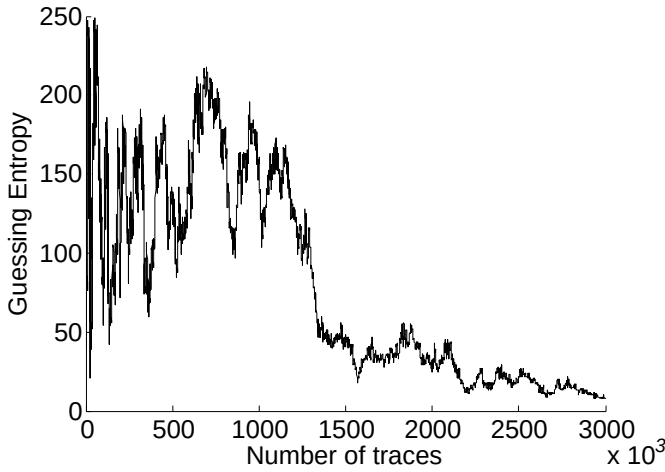
$$t = \frac{\mu(L_0) - \mu(L_1)}{\sqrt{\frac{\sigma^2(L_0)}{|L_0|} + \frac{\sigma^2(L_1)}{|L_1|}}} \quad (8)$$

where μ represents the expectation and σ^2 represents the sample variance among the set. The statistic t follows Student's t distribution under the null hypothesis. For the sake of simplicity, the threshold $|t| > 4.5$ for t is often used to reject

the null hypothesis, which corresponds to significance level $\alpha \leq 10^{-5}$. That means the two partition groups related to the sensitive variable can be distinguished, verifying the leakage.



(a) t -test under *BBI* model



(b) Guessing entropy

Fig. 11: Practical attack results

There are total 3,000,000 power traces are captured during the executions of RSM on FPGA. Then we launch the specific t -test for the measurement under *BBI* model. The intermediate value is set as the bit 1 of the output of the unmasked Sbox. The t value after analyzing 3,000,000 traces is shown in Fig. 11 (a). It can be found the result exceeds the threshold, which indicates the practical leakage of RSM's FPGA implementation.

Since it requires a large number of traces to perform a side-channel attack on the practical measurements, a more flexible metric named guessing entropy [32] is utilized to evaluate the average number of key candidates to test after the attack. The metric was proposed to quantify the effectiveness of side-channel attacks from the perspective of key entropy [33], [34].

We conduct the first-order attack on the set of 3,000,000 power traces. Firstly, we use the statistic alignment as the pre-

treatment for the power traces. Then we perform the first-order CPA under the bit 1 of the output of the Sbox. Fig. 11 (b) shows the guessing entropies under different number of traces. It can be found that the candidate space of the key decreases and tends to the minimum with increasing attack traces. The interesting thing is that launched attack is the first-order CPA against the unmasked Sbox output. It does not require the combining function in second-order or higher-order attacks. The bias among masked circuits makes the leakage of the circuit fail to achieve the bit balance.

V. COUNTERMEASURE

As shown in Proposition 1, the implementation of *BB* secure RSM scheme may not achieve first-order security on the FPGA structure. The imbalanced structure caused by uncontrolled EDA tools makes the theoretical secure scheme fail to achieve first-order security. It makes the circuit's power consumption still related to the sensitive value. In this section, we discuss possible countermeasures for the FPGA implementation of LEMS.

The goal here is to develop the implementation of masked Sbox with a consistent structure to eliminate the leakage. Since the maximum supported LUT is 6-input LUT in modern FPGA, the 256 addresses of the look-up table should be divided into 4 pieces so that each one can be implemented by 6-input LUTs. An exemplary design of the consistent circuit is shown in Fig. 12 (a). It can use the low 6 bits as the 6-input LUTs while the high 2 bits as the select signals for the multiplexers. It can avoid risk optimization during the synthesis of uncontrollable EDA tools. The bits used as the inputs for the 6-input LUTs and multiplexers are fixed in this way.

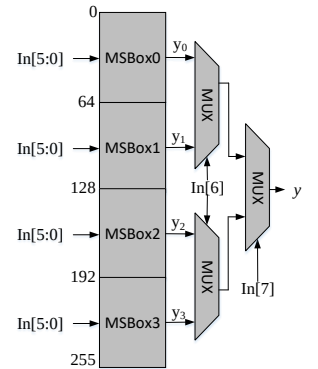
Moreover, the constraint (KEEP="TRUE") is applied to prevent optimizations in circuitual signals that are optimized or absorbed. The masked Sbox implementation generated by design is shown in Fig. 12 (b). The consistent structure of different masked Sboxes makes the circuit satisfy *BBI* security.

```

module MSBox(input[7:0] in, output reg[7:0] y);
(*KEEP="TRUE"*)wire [7:0] y0,y1,y2,y3;
MSBox0 m0(in[5:0],y0);
MSBox1 m1(in[5:0],y1);
MSBox2 m2(in[5:0],y2);
MSBox3 m3(in[5:0],y3);
always@(*) begin
case(in[7:6])
2'b00:y=y0;
2'b01:y=y1;
2'b10:y=y2;
2'b11:y=y3;
endcase
end
Endmodule

module MSBox0(input[5:0] in, output reg[7:0] s);
always@(*)
begin
case(in)
6'h00:s=8'h6C;
6'h01:s=8'h73;
...
6'h3F:s=8'h7A;
endcase
end
Endmodule

```



(a) Verilog code of the improved (b) Consistent implementation structure

Fig. 12: Modified design to avoid the leakage

Proposition 2 *In the circuit of i th-masked Sbox, the bits X_{LUT}^i are used as the input signals of 6-input LUT and X_{MUX}^i are used as the select signals for the multiplexors. If the implementation structures of 16 masked Sbox are consistent, i.e. $\forall i, j \in [0, 15], X_{LUT}^i = X_{LUT}^j, X_{MUX}^i = X_{MUX}^j$, the BB secure masking scheme achieves BBI security.*

Proof: The symbol definitions are consistent with the symbols used in the proof process of Proposition 1. $\forall i, j, X_{LUT}^i = X_{LUT}^j, X_{MUX}^i = X_{MUX}^j$. The BB secure masking scheme satisfies

$$\begin{aligned} E(t_{LUT}^0|X[k] = 0) &= E(t_{LUT}^0|X[k] = 1) \\ E(t_{LUT}^1|X[k] = 0) &= E(t_{LUT}^1|X[k] = 1) \\ E(t_{LUT}^2|X[k] = 0) &= E(t_{LUT}^2|X[k] = 1) \\ E(t_{LUT}^3|X[k] = 0) &= E(t_{LUT}^3|X[k] = 1) \end{aligned}$$

The toggles of multiplexors are connected to the output of LUTs and the select signals are the same among different masked Sboxes. Thus,

$$\begin{aligned} E(t_{MUX}^0|X[k] = 0) &= E(t_{MUX}^0|X[k] = 1) \\ E(t_{MUX}^1|X[k] = 0) &= E(t_{MUX}^1|X[k] = 1) \\ E(t_{MUX}^2|X[k] = 0) &= E(t_{MUX}^2|X[k] = 1) \end{aligned}$$

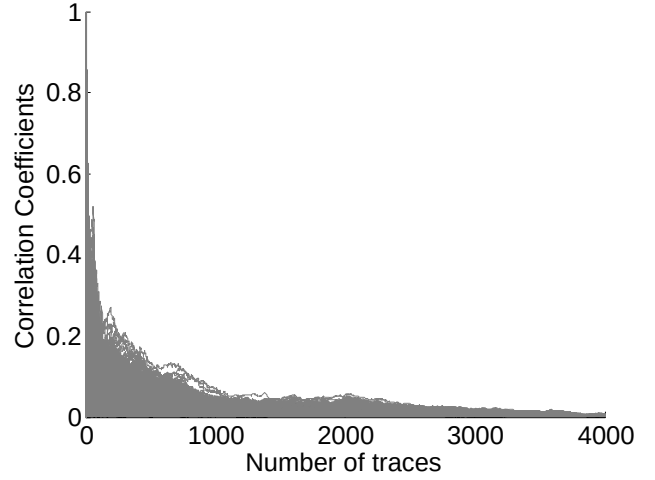
Thus, there has

$$E(H(X_M[k])|X[k] = 0) = E(H(X_M[k])|X[k] = 1)$$

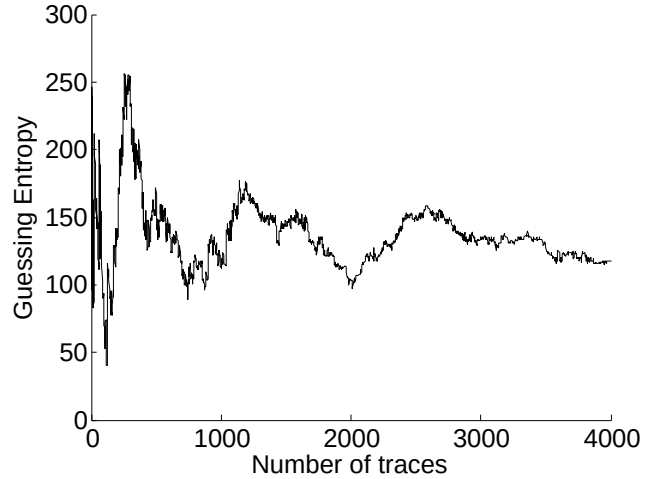
The results of applying first-order CPA on the modified implementation during simulation are shown in Fig. 13. We use the same number of traces as the experiment for the original implementation. In contrast to the results of the original flaw implementation, the correlation coefficient of the secret key is not visible among the candidates, as shown in Fig. 13 (a). We also compute the guessing entropy during the attack. It can be found that the guessing entropy tends to be stable with increasing the number of traces, shown in Fig. 13 (b). The secret key cannot be distinguished among the candidate keys efficiently.

We also conduct the practical attacks on the SAKURA-G board. The modified implementation is compiled and downloaded onto the board. The measurement setup is the same with the experiment of the original implementation of RSM. We perform the specific t -test on the 3,000,000 power traces. The intermediate value is set as the bit 1 of the output of the unmasked Sbox, which is the same with the above experiment. It can be observed that the result does not exceed the threshold in Fig. 14 (a). The first-order CPA is also performed on the trace set. Fig. 14 (b) shows the guessing entropies under different number of traces. The candidate space of the key tends to be stable at a high value. The power traces of the modified implementation do not support a successful first-order CPA attack for recovering the secret key.

Circuit optimization by Electronic Design Automation (EDA) tools would introduce extra threats to the implementation [21], [35]. During the design phase at different levels, the netlist includes not only the variables in the algorithm but also newly generated variables. Moreover, some details may



(a) Correlation coefficients



(b) Guessing entropy

Fig. 13: Attack results on the simulation power traces of the modified implementation

be realized unexpectedly. It is because that optimization aims to improve speed or area rather than security. Some options can be selected to set the design goal and strategy. For example, Xilinx supports the setup to limit the LUT-FF pairs utilization ratio and LUT combining. Generally, the supported strategies are limited and coarse-grained. Knowing the cause of the leakage is a challenge for designers to implement a scheme securely. Thus, it often needs the pre-silicon evaluation to detect the leakage of the cryptographic circuits before manufacture.

During the pre-silicon design phase, there are two ways to leakage detection. The first type is to perform static analysis based on power simulation. It can perform the leakage testing methodology (TVLA) or power analysis attacks (DPA, CPA, et al.) on the simulation results of different levels [36], [37]. Another type is formal verification, which proves the scheme's security by formal method at the algorithm level [38], [39], [40], [41]. Most of them are integrated with the provable security model for the higher-order masking scheme. It is

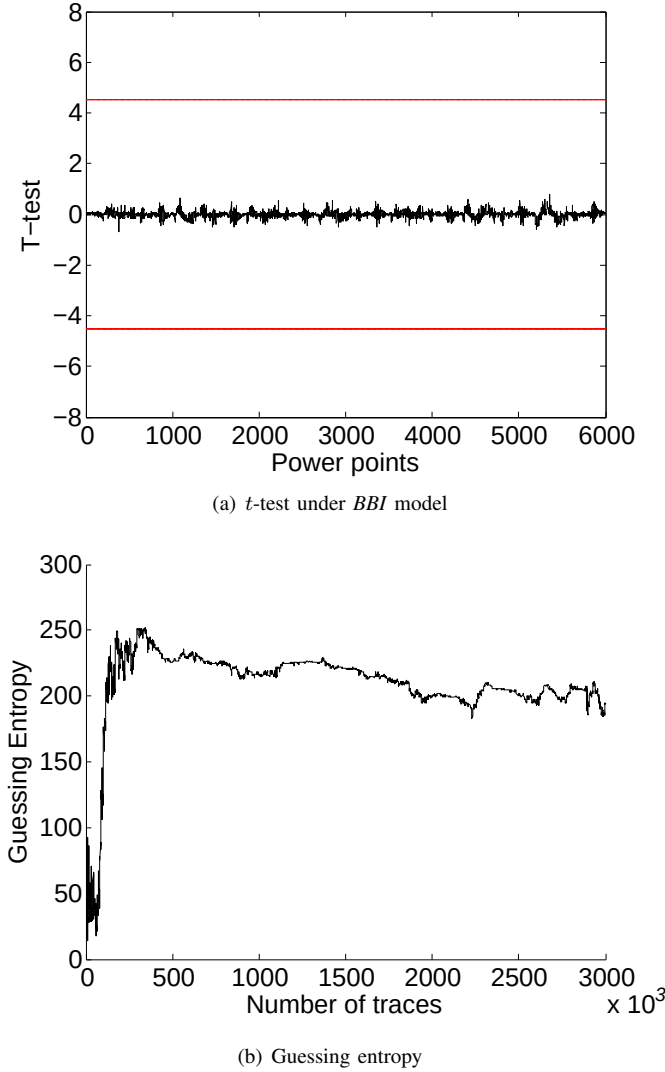


Fig. 14: Practical attack results on the modified implementation

not suitable for LEMS. In addition, some schemes verify the masked hardware implementation in the presence of glitches. For example, REBECCA estimates the security of the circuits under the probing model for the worst-case signal timings [41]. It does not cover the structure factor at the implementation level.

The proposed security model can be used as a supplement for the above two types of methods. Firstly, BB security model is the essential condition to resist first-order attacks. It can be used at the algorithm level to detect the flaws of the masking scheme. Then, BBI security model can be integrated into the EDA process to evaluate the security of the implementation of LEMS. After detecting the leakage in the design of different levels, the designer needs to modify their design or implementation to remove the leakage. If the detected leakage is introduced by the EDA optimization, like the inconsistency architecture flaw, the designer could try to remove the leakage by changing the optimization strategy. The designer can add the constraints to disable the optimization

and eliminate leakage. For example, the command (e.g., KEEP TRUE) can constrain the crucial signals in the implementation. It can prevent crucial signals from being modified or removed during optimization. It is necessary to feedback to the former security evaluations after modification until there is no leakage in the whole period of design.

VI. DISCUSSION

A. RSM variants

RSM and its variants are typical of the LEMS implementations. It consumes a small amount of area and performance by limiting the value range of the random mask to 16 fixed elements. In order to guarantee security, it is necessary to achieve BBI security when choosing a random mask in the fixed masking set. Currently, there are four mask sets in RSM and its variants, shown in Equation (4). The latest scheme with $MaskSet_4$ proposed by Moradi et al. [14] had not been attacked, which is our target during the above analysis. Hereafter, we analyze the RSM scheme using the mask set $MaskSet_1$ (RSM v1) [9], RSM variant used mask set $MaskSet_2$ (RSM v2) [11], RSM variant used mask set $MaskSet_3$ (RSM v3) [26].

To analyze the leakage of the implementations, we synthesize the implementations and obtain the netlists at the place & route level. The experimental environment is the same as in Section IV. In the netlist, the structures of 16 masked Sboxes are also inconsistent. The structures of the 2nd-byte and 13th-byte masked Sboxes differ from others. The difference of conditional expectation (DCE) is shown in Fig. 15. The results show that bias exists for each bit under the three schemes.

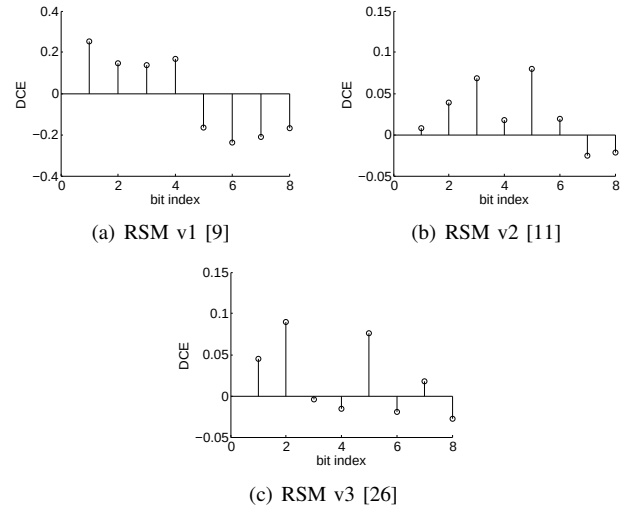


Fig. 15: Analysis of RSM and its variants.

To prevent inconsistent structures, we replace the masked SBox with the modified design to avoid leakage. We fix the low six bits as the 6-input LUTs while the high two bits are the select signals for the multiplexors. We also add the constraint (KEEP="TRUE") to prevent optimizations in circuitual signals that are optimized or absorbed. The obtained netlists show that the structures of different masked Sboxes are consistent.

B. FPGA families

Xilinx has developed more advanced 7 Series FPGAs to satisfy various application needs. The series includes sub-families Artix-7, Kintex-7, Virtex-7, and so on. Unlike previous families, the 7 Series FPGAs utilize a unified 28nm design architecture while ensuring a consistent user experience and eliminating the inconvenience of switching between them.

We also analyze the target implementation of these advanced families. The setting is the same as the experiments on Spartan-6. The results are shown in Table I. It can be observed that the inconsistent structure of the masked SBox implementation exists in these FPGA series. The input and select signals are the same as the implementation on Spartan-6, leading to the same bias. The improved implementations of the countermeasure are conducted in the FPGAs. The results show that the structures are consistent in the netlist.

C. EDA software

We also discuss the results of various EDA software. Vivado was developed for next-generation devices, including UltraScale, UltraScale+, and Zynq-7000 MPSoCs, and integrates a unified design environment leveraging high-level synthesis (HLS), IP-centric workflows and advanced algorithmic optimizations. Vivados parallelized implementation algorithms reduce compile times by approximately 30 ~ 50% compared to ISE for equivalent designs, while its hierarchical floor-planning tools improve resource utilization by 15 ~ 20%. However, ISE remains prevalent in legacy system maintenance due to its lightweight footprint and compatibility with obsolete toolchains.

We analyze the target implementation on vivado2012. The compiled netlist shows that the structure of the masked SBox is inconsistent, and the $DCE = 0.0371$ indicates the vulnerabilities in the implementation. We also conduct experiments on the latest version, vivado2024. The results show that the structure of masked SBox is consistent. The result $DCE = 0$ indicates the implementation is BBI secure. This inconsistent structure may be related to the synthesis strategy optimization algorithms embedded in EDA software. We use the default synthesis strategy in the software, which provides 7 * 35 optimization strategy combinations for synthesis strategy and implementation strategy optimization. Analyzing the relationship between this leakage and optimization algorithms is one of our future research work.

Additionally, we conduct experiments using Yosys, an open-source Verilog-based hardware synthesis and verification framework. It employs a modular architecture centered on its internal RTL Intermediate Language, enabling cross-platform synthesis for FPGA and ASIC targets. We synthesize the design with the latest version, Yosys-0.51. There are also vulnerabilities in the implementation of the compiled netlist. We also experiment with the countermeasure of this software. The results are shown in Table I.

VII. CONCLUSION

LEMS is one of the most popular lightweight countermeasures against SCAs. This paper has highlighted the leakage of

TABLE I: Analysis on RSM variants, FPGA families and EDA softwares.

RSM variants			
	DCE(max)	BBI	countermeasure
RSM v1	0.2539	✗	✓
RSM v2	0.0801	✗	✓
RSM v3	0.0898	✗	✓
RSM v4	0.0371	✗	✓
FPGA families			
	DCE(max)	BBI	countermeasure
Spartan-6	0.0371	✗	✓
Artix-7	0.0371	✗	✓
Virtex-7	0.0371	✗	✓
Kintex-7	0.0371	✗	✓
EDA software			
	DCE(max)	BBI	countermeasure
ISE14.7	0.0371	✗	✓
vivado2012	0.0371	✗	✓
vivado2021	0	✓	✓
Yosys-0.51	0.0703	✗	✓

LEMS in FPGA implementation. We extend *BB* secure model to *BBI* secure model at the hardware implementation level. The security models can be used to evaluate the security of LEMS in a sample way before practical leakage assessment. Moreover, it helps to improve the security of implementation during the design phase.

Based on the definitions of the security model and proposition, we theoretically prove LEMS's vulnerability in FPGA implementation. The FPGA design differs from the ASIC counterpart, most notably in using LUTs. It affects the implementation cost by the number of inputs to a Boolean function more seriously than algebraic complexity. We have analyzed the security of FPGA primitives to implement a countermeasure based on RSM. The imbalanced masking caused by uncontrolled EDA tools makes the circuit's power consumption still related to the sensitive variable. The vulnerability of imbalanced architecture makes the RSM implementation fail to resist the first-order attack under *BBI* model. Our proof and experiments suggest that the implementation of the masking scheme needs security evaluation or testing during the design phase. It can be performed to apply for the netlist based on the provided security model, which can also be integrated into the existing EDA tools.

ACKNOWLEDGMENTS

This work was supported in part by the National Key R&D Program of China (2023YFB3106800), National Natural Science Foundation of China under Grant (NO.62472259, NO.62072247, NO.62072398, NO.62172258), the Program of Taishan Young Scholars of Shandong Province, project ZR2024QF098 supported by Shandong Provincial Natural Science Foundation.

REFERENCES

- [1] Y. Ishai, A. Sahai, and D. Wagner, "Private circuits: Securing hardware against probing attacks," in *Proc. Annu. Int. Cryptol. Conf.*, 2003, pp. 463–481.
- [2] M. Rivain and E. Prouff, "Provably secure higher-order masking of aes," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2010, pp. 413–427.

- [3] J.-S. Coron, E. Prouff, M. Rivain, and T. Roche, "Higher-order side channel security and mask refreshing," in *Proc. Int. Workshop Softw. Fast Softw. Encry.*, 2013, pp. 410–424.
- [4] H. Groß, S. Mangard, and T. Korak, "Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order," in *Proc. ACM Workshop Theory Implement. Sec.*, 2016, p. 3.
- [5] H. Groß, S. Mangard, and T. Korak, "An efficient side-channel protected aes implementation with arbitrary protection order," in *Proc. Top. Cryptol. CT-RSA 2017*, 2017, pp. 95–112.
- [6] R. Ueno, N. Homma, and T. Aoki, "Toward more efficient dpa-resistant aes hardware architecture based on threshold implementation," in *Proc. Int. Workshop Constr. SCA Secure Des.*, 2017, pp. 50–64.
- [7] M. Nassar, Y. Souissi, S. Guilley, and J.-L. Danger, "Rsm: a small and fast countermeasure for aes, secure against 1st and 2nd-order zero-offset scas," in *Proc. 2012 Des. Autom. Test Eur. Conf. Exhib.*, 2012, pp. 1173–1178.
- [8] N. Yamashita, K. Minematsu, T. Okamura, and Y. Tsunoo, "A smaller and faster variant of rsm," in *Proc. 2014 Des. Autom. Test Eur. Conf. Exhib.*, 2014, pp. 1–6.
- [9] S. Bhasin, N. Bruneau, J.-L. Danger, S. Guilley, and Z. Najm, "Analysis and improvements of the dpa contest v4 implementation," in *Proc. Int. Conf. Sec. Priv. Appl. Cryptogr. Eng.*, 2014, pp. 201–218.
- [10] X. Ye and T. Eisenbarth, "On the vulnerability of low entropy masking schemes," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2013, pp. 44–60.
- [11] Z. Liu, N. Gao, C. Tu, Y. Ma, and Z. Liu, "Detecting side channel vulnerabilities in improved rotating s-box masking schemes presenting four non-profiled attacks," in *Proc. Int. Conf. Sel. Areas Cryptogr.*, 2016, pp. 41–57.
- [12] F. Zhang, L. Geng, J. Shen, S. Bhasin, X. Zhao, and S. Guo, "Improved low-entropy masking scheme for LED with mitigation against correlation-enhanced collision attacks," in *Proc. Asian Hardware Oriented Sec. Trust. Symp.*, 2017, pp. 49–54.
- [13] J. Ming, Y. Zhou, W. Cheng, H. Li, G. Yang, and Q. Zhang, "Mind the balance: Revealing the vulnerabilities in low entropy masking schemes," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3694–3708, 2020.
- [14] A. Moradi, S. Guilley, and A. Heuser, "Detecting hidden leakages," in *Proc. Int. Conf. Appl. Cryptogr. Netw. Secur.*, 2014, pp. 324–342.
- [15] V. Grosso, F.-X. Standaert, and E. Prouff, "Low entropy masking schemes, revisited," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2013, pp. 33–43.
- [16] A. Moradi and V. Immler, "Early propagation and imbalanced routing, how to diminish in fpgas," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2014, pp. 598–615.
- [17] P. Sasdrich, O. Mischke, A. Moradi, and T. Güneysu, "Side-channel protection by randomizing look-up tables on reconfigurable hardware," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2015, pp. 95–107.
- [18] S. Mangard, T. Popp, and B. M. Gammel, "Side-channel leakage of masked cmos gates," in *Proc. Top. Cryptol. CT-RSA 2005*, 2005, pp. 351–365.
- [19] S. Mangard and K. Schramm, "Pinpointing the side-channel leakage of masked aes hardware implementations," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2006, pp. 76–90.
- [20] S. Mangard, N. Pramstaller, and E. Oswald, "Successfully attacking masked aes hardware implementations," in *Proc. Int. Workshop Cryptogr. Hardw. Embed. Syst.*, 2005, pp. 157–171.
- [21] D. B. Roy, S. Bhasin, S. Guilley, J.-L. Danger, and D. Mukhopadhyay, "From theory to practice of private circuit: A cautionary note," in *Proc. 2015 33rd IEEE Int. Conf. Comput. Des.*, 2015, pp. 296–303.
- [22] Y. Li, Z. Liu, S. Guilley, and M. Tang, "Analysis of multiplicative low entropy masking schemes against correlation power attack," *IEEE Trans. Inf. Forensics Secur.*, vol. 16, pp. 4466–4481, 2021.
- [23] W. Kai and X. Zhiwei, "Synopsys prime power manual release u-2003.06-qa," 2003.
- [24] T. Zhang, J. Park, M. Tehranipoor, and F. Farahmandi, "Psc-tg: Rtl power side-channel leakage assessment with test pattern generation," in *Proc. 2021 58th ACM/IEEE Des. Autom. Conf.*, 2021, pp. 709–714.
- [25] M. Gruber and G. Sigl, "Tofu-toggle count analysis made simple," *Cryptology ePrint Archive*, 2022.
- [26] N. Veshchikov and S. Guilley, "Implementation flaws in the masking scheme of dpa contest v4," *IET Information Security*, vol. 11, no. 6, pp. 356–362, 2017.
- [27] S. Bhasin, C. Carlet, and S. Guilley, "Theory of masking with code-words in hardware: low-weight dth-order correlation-immune boolean functions," *IACR Cryptol. ePrint Arch.*, vol. 2013, p. 303, 2013.
- [28] A. Nadjia and A. Mohamed, "Efficient implementation of aes s-box in lut-6 fpgas," in *Proc. 2015 4th Int. Conf. on Elect. Eng.*, 2015, pp. 1–4.
- [29] H. Guntur, J. Ishii, and A. Satoh, "Side-channel attack user reference architecture board sakura-g," in *Proc. 2014 IEEE 3rd Global Conf. Consum. Electron.*, 2014, pp. 271–274.
- [30] B. J. Gilbert Goodwill, J. Jaffe, P. Rohatgi *et al.*, "A testing methodology for side-channel resistance validation," in *Proc. NIST non-invasive attack Test. Workshop*, 2011, pp. 115–136.
- [31] T. Schneider and A. Moradi, "Leakage assessment methodology," *J. Cryptogr. Eng.*, vol. 6, no. 2, pp. 85–99, 2016.
- [32] F.-X. Standaert, T. G. Malkin, and M. Yung, "A unified framework for the analysis of side-channel key recovery attacks," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2009, pp. 443–461.
- [33] J. L. Massey, "Guessing and entropy," in *Proc. IEEE Int. Symp. Inf. Theory*, 1994, p. 204.
- [34] B. Köpf and D. Basin, "An information-theoretic model for adaptive side-channel attacks," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2007, pp. 286–296.
- [35] J.-L. Danger, S. Guilley, P. Nguyen, R. Nguyen, and Y. Souissi, "Analyzing security breaches of countermeasures throughout the refinement process in hardware design flow," in *Proc. Des. Autom. Test Eur.*, 2017, pp. 1129–1134.
- [36] A. A. Ding, L. Zhang, F. Durvaux, F.-X. Standaert, and Y. Fei, "Towards sound and optimal leakage detection procedure," in *Proc. Int. Conf. Smart Card Res. Adv. Appl.*, 2017, pp. 105–122.
- [37] A. Moradi, B. Richter, T. Schneider, and F.-X. Standaert, "Leakage detection with the x2-test," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 1, pp. 209–237, 2018.
- [38] G. Barthe, S. Belaid, F. Dupressoir, P.-A. Fouque, B. Grégoire, and P.-Y. Strub, "Verified proofs of higher-order masking," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2015, pp. 457–485.
- [39] S. Faust, V. Grosso, S. M. Del Pozo, C. Paglialonga, and F.-X. Standaert, "Composable masking schemes in the presence of physical defaults & the robust probing model," *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, vol. 2018, no. 3, pp. 89–120, 2018.
- [40] G. Barthe, S. Belaid, P. Fouque, and B. Gregoire, "maskverif: a formal tool for analyzing software and hardware masked implementations," *IACR Cryptol. ePrint Arch.*, vol. 2018, p. 562, 2018.
- [41] R. Bloem, H. Gross, R. Iusupov, B. Könnighofer, S. Mangard, and J. Winter, "Formal verification of masked hardware implementations in the presence of glitches," in *Proc. Annu. Int. Conf. Theory Appl. Cryptogr. Tech.*, 2018, pp. 321–353.



Yanbin Li received the Ph.D degree in School of Cyber Science and Engineering from Wuhan University. He is an associate professor at the School of Software, Shandong University, and with the State Key Laboratory of Cryptology, P.O.Box 5159. His research interests include side-channel analysis and cryptanalysis.



Yikang Guo is studying for a master's degree in Electronic Information at Nanjing Agricultural University. His research interests include side-channel analysis and cryptanalysis.



Fan Zhang (Member, IEEE) received his Ph.D. degree from the Department of Computer Science and Engineering, University of Connecticut, CT, USA, in 2011. He is currently a Full Professor with the College of Computer Science and Technology, Zhejiang University, Hangzhou, China, and also with the Alibaba-zhejiang University Joint Institute of Frontier Technologies, Hangzhou. His research interests include system security, hardware security, network security, cryptography, and computer architecture.